Escuela Superior Politécnica Agropecuaria de Manabí

Informe Portafolio Técnico Final de la Unidad 1

Carrera: Desarrollo de Software

Materia: Estructuras de Datos y Algoritmos

Docente: Luiggi Alexander Jalca Saltos

Estudiante: Geovanny Francisco Ríos Rodríguez

Caso Asignado: Lista de Reproducción Multimedia con Bucle Continuo (Playlist)
Fecha: lunes, 12 de octubre de 2026

Enlace Repositorio de GitHub: https://github.com/riroguez-espam/EDA-2N-ESPAM-GFRR

1. OBJETIVO PEDAGÓGICO DEL PORTAFOLIO

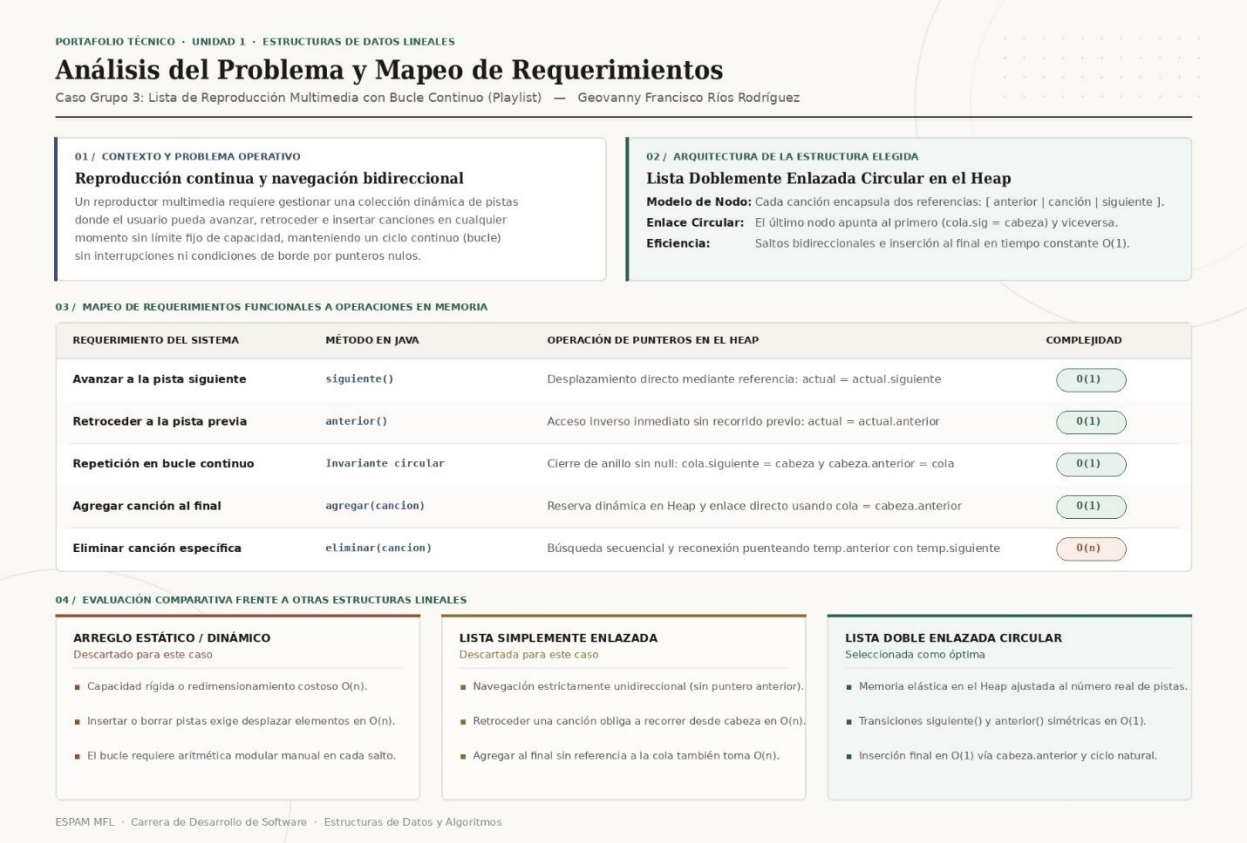
El presente Portafolio Técnico de Ingeniería de Software tiene como propósito demostrar de forma práctica, rigurosa y verificable el dominio conceptual, analítico y algorítmico de las Estructuras de Datos Lineales fundamentales desarrolladas durante la Unidad 1 de la asignatura Estructuras de Datos y Algoritmos. El proyecto consolida la implementación manual al 100% en lenguaje Java —sin depender de colecciones preconstruidas del lenguaje— de arreglos estáticos unidimensionales y matrices bidimensionales, listas simplemente enlazadas en memoria dinámica, listas doblemente enlazadas circulares, pilas bajo política LIFO (Last-In, First-Out) y colas circulares bajo política FIFO (First-In, First-Out) con aritmética modular.

Asimismo, este portafolio articula la fundamentación teórica con la resolución del caso de estudio del mundo real asignado al Grupo 3 en el Foro de Debate Técnico de la Clase 06: una Lista de Reproducción Multimedia con Bucle Continuo (Playlist). A través de esta implementación se contrasta el desempeño de estructuras contiguas frente a estructuras dinámicas basadas en nodos enlazados en el Heap, evaluando el impacto del direccionamiento de referencias, la ausencia de punteros nulos en anillos circulares y el cumplimiento estricto de complejidades temporales constantes O(1) frente a costos lineales O(n) en la gestión de memoria Stack y Heap de la Máquina Virtual de Java (JVM).

2. EVIDENCIAS GRÁFICAS Y ANALÍTICAS COMENTADAS POR FASE

2.1. Fase 01: Análisis del Problema (Caso Asignado en el Foro — Grupo 3)

En la primera fase se aborda el levantamiento de requerimientos funcionales y técnicos para el caso asignado al Grupo 3: Lista de Reproducción Multimedia con Bucle Continuo (Playlist). Un reproductor moderno exige tres operaciones críticas en tiempo real: avanzar a la pista siguiente, retroceder a la pista previa y repetir el catálogo completo en bucle continuo sin interrupciones, además de permitir la adición dinámica de nuevas canciones sin un límite rígido de capacidad.



PORTAFOLIO TÉCNICO · UNIDAD 1 · ESTRUCTURAS DE DATOS LINEALES

Análisis del Problema y Mapeo de Requerimientos

Caso Grupo 3: Lista de Reproducción Multimedia con Bucle Continuo (Playlist) — Geovanny Francisco Ríos Rodríguez

01 / CONTEXTO Y PROBLEMA OPERATIVO

Reproducción continua y navegación bidireccional

Un reproductor multimedia requiere gestionar una colección dinámica de pistas donde el usuario pueda avanzar, retroceder e insertar canciones en cualquier momento sin límite fijo de capacidad, manteniendo un ciclo continuo (bucle) sin interrupciones ni condiciones de borde por punteros nulos.

02 / ARQUITECTURA DE LA ESTRUCTURA ELEGIDA

Lista Doblemente Enlazada Circular en el Heap

Modelo de Nodo: Cada canción encapsula dos referencias: [anterior | canción | siguiente].

Enlace Circular: El último nodo apunta al primero (cola.sig = cabeza) y viceversa.

Eficiencia: Saltos bidireccionales e inserción al final en tiempo constante O(1).

03 / MAPEO DE REQUERIMIENTOS FUNCIONALES A OPERACIONES EN MEMORIA

REQUERIMIENTO DEL SISTEMA	MÉTODO EN JAVA	OPERACIÓN DE PUNTEROS EN EL HEAP	COMPLEJIDAD
Avanzar a la pista siguiente	`siguiente()`	Desplazamiento directo mediante referencia: actual = actual.siguiente	O(1)
Retroceder a la pista previa	`anterior()`	Acceso inverso inmediato sin recorrido previo: actual = actual.anterior	O(1)
Repetición en bucle continuo	`Invariante circular`	Cierre de anillo sin null: cola.siguiente = cabeza y cabeza.anterior = cola	O(1)
Agregar canción al final	`agregar(cancion)`	Reserva dinámica en Heap y enlace directo usando cola = cabeza.anterior	O(1)
Eliminar canción específica	`eliminar(cancion)`	Búsqueda secuencial y reconexión puenteando temp.anterior con temp.siguiente	O(n)

04 / EVALUACIÓN COMPARATIVA FRENTE A OTRAS ESTRUCTURAS LINEALES

ARREGLO ESTÁTICO / DINÁMICO
Descartado para este caso

- Capacidad rígida o redimensionamiento costoso O(n).
- Insertar o borrar pistas exige desplazar elementos en O(n).
- El bucle requiere aritmética modular manual en cada salto.

LISTA SIMPLEMENTE ENLAZADA
Descartada para este caso

- Navegación estrictamente unidireccional (sin puntero anterior).
- Retroceder una canción obliga a recorrer desde cabeza en O(n).
- Agregar al final sin referencia a la cola también toma O(n).

LISTA DOBLE ENLAZADA CIRCULAR
Seleccionada como óptima

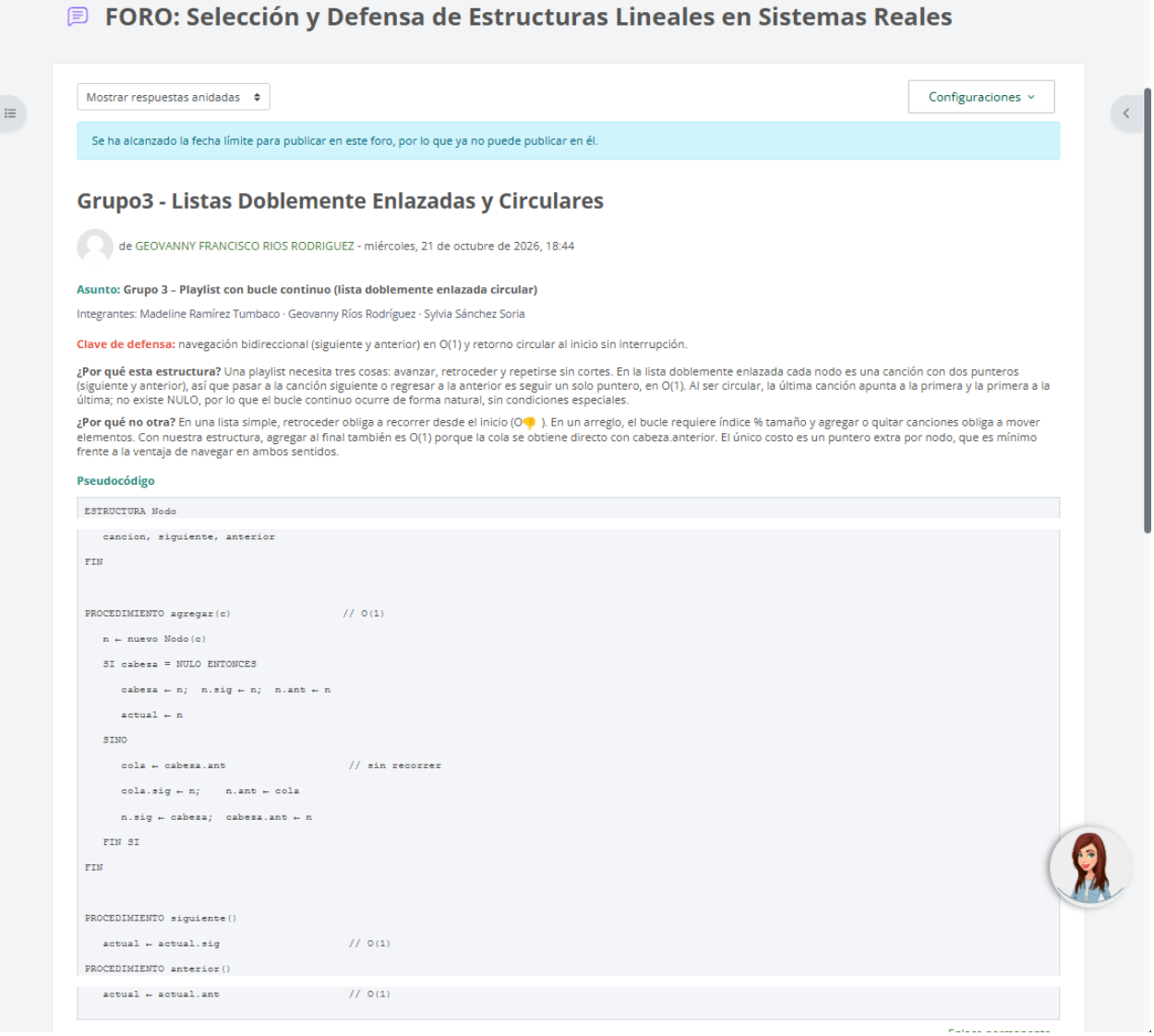
- Memoria elástica en el Heap ajustada al número real de pistas.
- Transiciones siguiente() y anterior() simétricas en O(1).
- Inserción final en O(1) vía cabeza.anterior y ciclo natural.

ESPAM MFL · Carrera de Desarrollo de Software · Estructuras de Datos y Algoritmos

Análisis técnico de requerimientos: La Lista Doblemente Enlazada Circular resuelve de manera óptima estos requerimientos porque cada canción se encapsula en un objeto Nodo alojado dinámicamente en el Heap con dos referencias explícitas (siguiente y anterior). A diferencia de una lista simplemente enlazada —donde retroceder a la canción previa obliga a recorrer desde la cabeza en tiempo lineal O(n)— o de un arreglo estático —donde insertar o eliminar pistas exige desplazar bloques contiguos en O(n)—, la estructura doblemente enlazada circular ejecuta tanto el salto hacia adelante como el salto hacia atrás en tiempo constante O(1), manteniendo el bucle infinito de forma nativa al enlazar la cola con la cabeza sin punteros null.



Sustentación mediante técnica SCQA: Mediante la metodología SCQA se sintetiza el problema arquitectónico: en la Situación (S), los sistemas de audio requieren listas dinámicas con navegación fluida y repetición continua; en la Complicación (C), las estructuras contiguas sufren sobrecosto O(n) al redimensionar o desplazar elementos y las listas simples impiden retroceder en O(1); ante la Pregunta (Q) de cómo garantizar transiciones bidireccionales, inserciones al final y bucle continuo en O(1), la Respuesta (A) demuestra que la Lista Doblemente Enlazada Circular elimina los casos de borde y permite obtener la cola directamente mediante cabeza.anterior en O(1).



FORO: Selección y Defensa de Estructuras Lineales en Sistemas Reales

Mostrar respuestas anidadas

Configuraciones

Se ha alcanzado la fecha límite para publicar en este foro, por lo que ya no puede publicar en él.

Grupo3 - Listas Doblemente Enlazadas y Circulares

de GEOVANNY FRANCISCO RIOS RODRIGUEZ - miércoles, 21 de octubre de 2026, 18:44

Asunto: Grupo 3 – Playlist con bucle continuo (lista doblemente enlazada circular)

Integrantes: Madeline Ramírez Tumbaco · Geovanny Ríos Rodríguez · Sylvia Sánchez Soria

Clave de defensa: navegación bidireccional (siguiente y anterior) en O(1) y retorno circular al inicio sin interrupción.

¿Por qué esta estructura? Una playlist necesita tres cosas: avanzar, retroceder y repetirse sin cortes. En la lista doblemente enlazada cada nodo es una canción con dos punteros (siguiente y anterior), así que pasar a la canción siguiente o regresar a la anterior es seguir un solo puntero, en O(1). Al ser circular, la última canción apunta a la primera y la primera a la última; no existe NULO, por lo que el bucle continuo ocurre de forma natural, sin condiciones especiales.

¿Por qué no otra? En una lista simple, retroceder obliga a recorrer desde el inicio (O👎). En un arreglo, el bucle requiere índice % tamaño y agregar o quitar canciones obliga a mover elementos. Con nuestra estructura, agregar al final también es O(1) porque la cola se obtiene directo con cabeza.anterior. El único costo es un puntero extra por nodo, que es mínimo frente a la ventaja de navegar en ambos sentidos.

Pseudocódigo

```
ESTRUCTURA Nodo
   cancion, siguiente, anterior
FIN

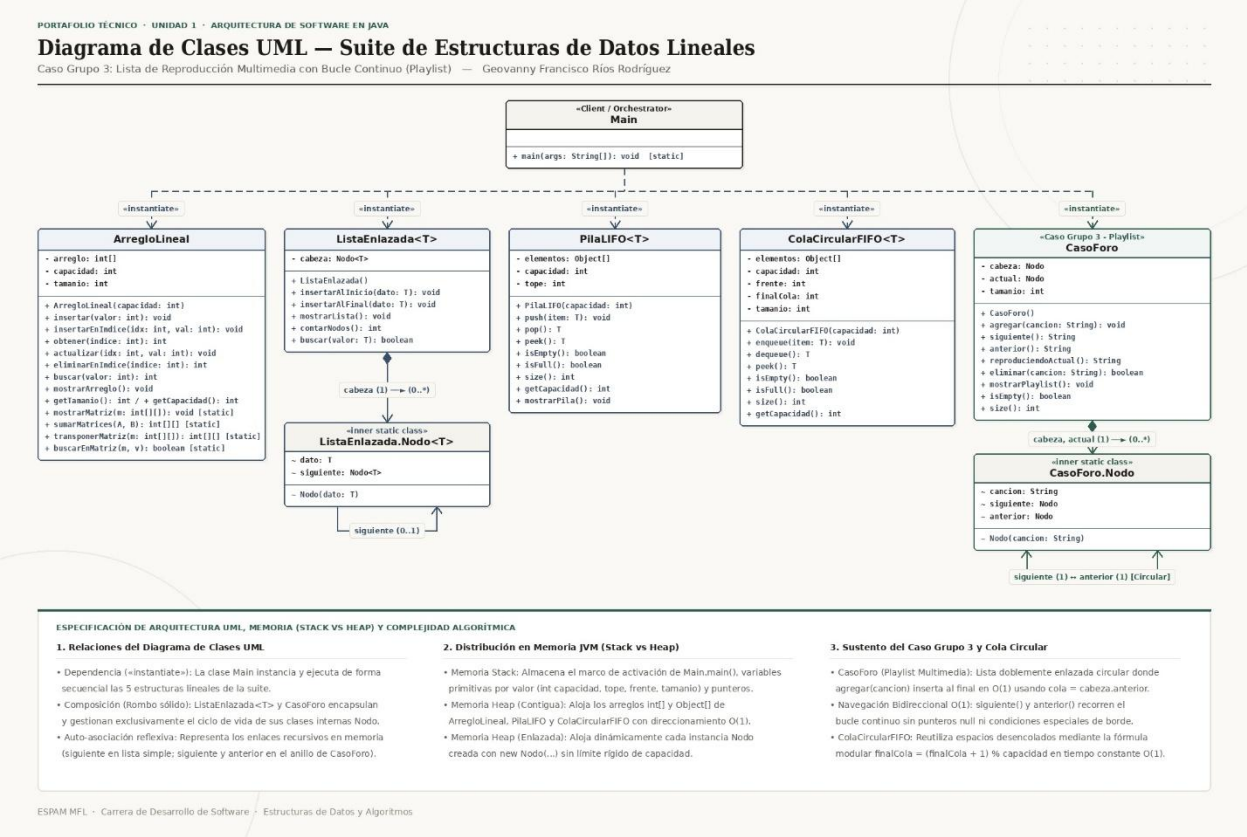

PROCEDIMIENTO agregar(c)                  // O(1)
   n ← nuevo Nodo(c)
   SI cabeza = NULO ENTONCES
      cabeza ← n;  n.sig ← n;  n.ant ← n
      actual ← n
   SINO
      cola ← cabeza.ant                   // sin recorrer
      cola.sig ← n;    n.ant ← cola
      n.sig ← cabeza;  cabeza.ant ← n
   FIN SI
FIN


PROCEDIMIENTO siguiente()
   actual ← actual.sig                    // O(1)
PROCEDIMIENTO anterior()
   actual ← actual.ant                    // O(1)
```

Análisis de la participación en el Foro Moodle: En la defensa técnica presentada en el Foro de la Clase 06 se sustentó el diseño algorítmico del nodo con triple campo (cancion, siguiente, anterior) y el procedimiento de inserción al final en O(1) sin recorrer la estructura, aprovechando la propiedad circular donde cabeza.anterior referencia permanentemente al último nodo insertado.

2.2. Fase 02: Diagramas de Arquitectura y Memoria (Stack vs. Heap)

En la segunda fase se modela la arquitectura de software de la suite completa en Java mediante un Diagrama de Clases UML elaborado en Draw.io, especificando la relación entre el programa principal orquestador (Main), las estructuras lineales contiguas y enlazadas, sus clases internas Nodo y la distribución de memoria entre el Stack y el Heap.

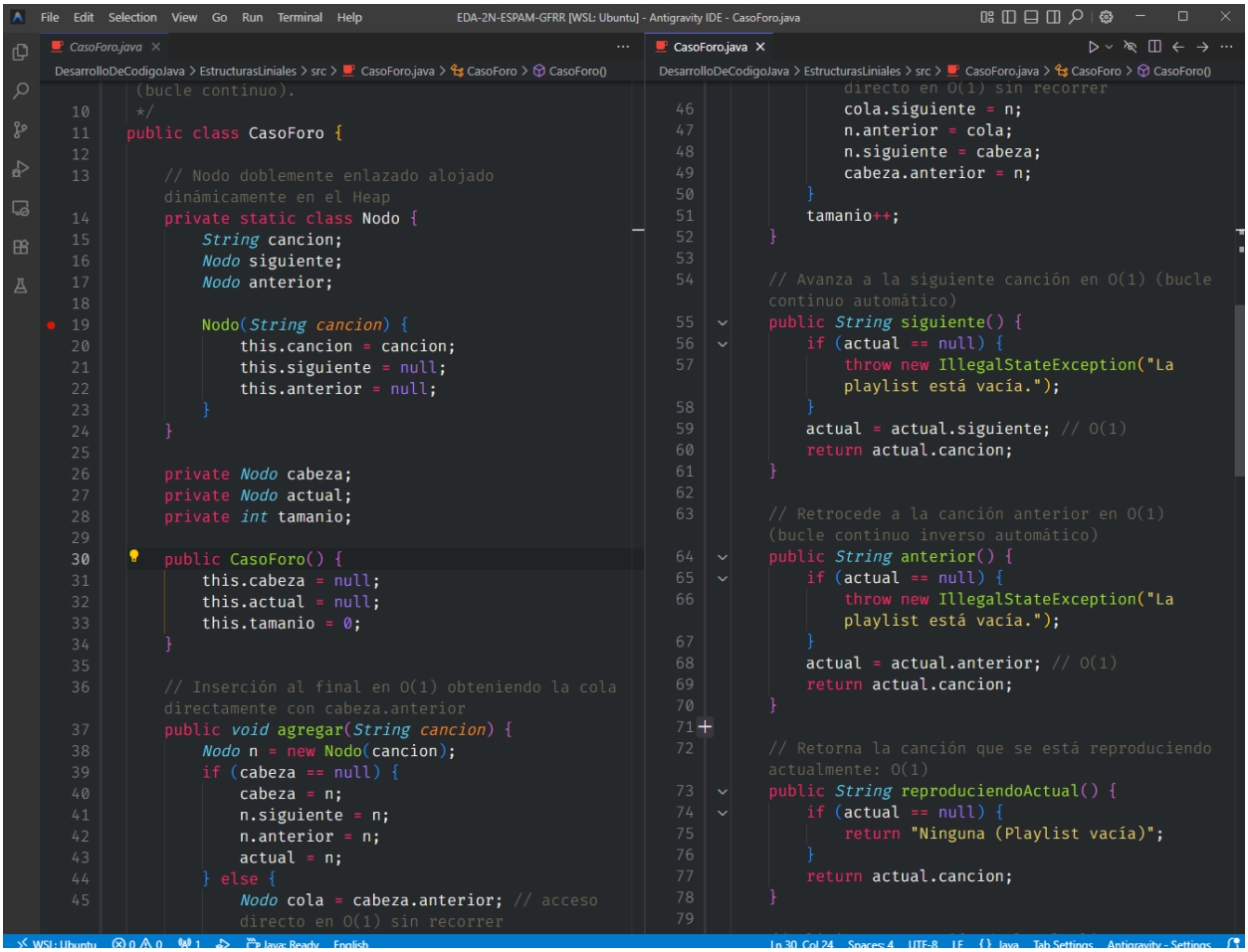


Sustentación de Arquitectura y Memoria (Stack vs. Heap): El diseño arquitectónico refleja con precisión cómo la Máquina Virtual de Java (JVM) administra la memoria durante la ejecución de la suite: en el Stack (pila de llamadas LIFO) se alojan el marco de activación del método main(), las variables primitivas por valor (como capacidad, tamanio, frente, finalCola y tope) y las variables de referencia locales (arreglo, lista, pila, cola y playlist). Por el contrario, en el Heap (memoria dinámica) se almacenan tanto los bloques contiguos de memoria de los arreglos internos (int[] y Object[]) como cada instancia individual de Nodo creada con el operador new en ListaEnlazada y CasoForo, enlazadas mediante referencias de memoria sin requerir contigüidad física.

2.3. Fase 03: Desarrollo de Código JAVA (Suite de Estructuras Lineales)

A continuación, se presenta y documenta la implementación manual en Java de cada módulo dentro del directorio EstructurasLineales/src/, detallando el funcionamiento algorítmico y la complejidad temporal de sus métodos principales.

A) Módulo del Caso del Foro (CasoForo.java): Implementa la Lista de Reproducción Multimedia con Bucle Continuo asignada al Grupo 3 mediante una Lista Doblemente Enlazada Circular en el Heap.



```
(bucle continuo).
 */
public class CasoForo {

    // Nodo doblemente enlazado alojado
    dinámicamente en el Heap
    private static class Nodo {
        String cancion;
        Nodo siguiente;
        Nodo anterior;

        Nodo(String cancion) {
            this.cancion = cancion;
            this.siguiente = null;
            this.anterior = null;
        }
    }

    private Nodo cabeza;
    private Nodo actual;
    private int tamanio;

    public CasoForo() {
        this.cabeza = null;
        this.actual = null;
        this.tamanio = 0;
    }

    // Inserción al final en O(1) obteniendo la cola
    directamente con cabeza.anterior
    public void agregar(String cancion) {
        Nodo n = new Nodo(cancion);
        if (cabeza == null) {
            cabeza = n;
            n.siguiente = n;
            n.anterior = n;
            actual = n;
        } else {
            Nodo cola = cabeza.anterior; // acceso
            directo en O(1) sin recorrer
            cola.siguiente = n;
            n.anterior = cola;
            n.siguiente = cabeza;
            cabeza.anterior = n;
        }
        tamanio++;
    }

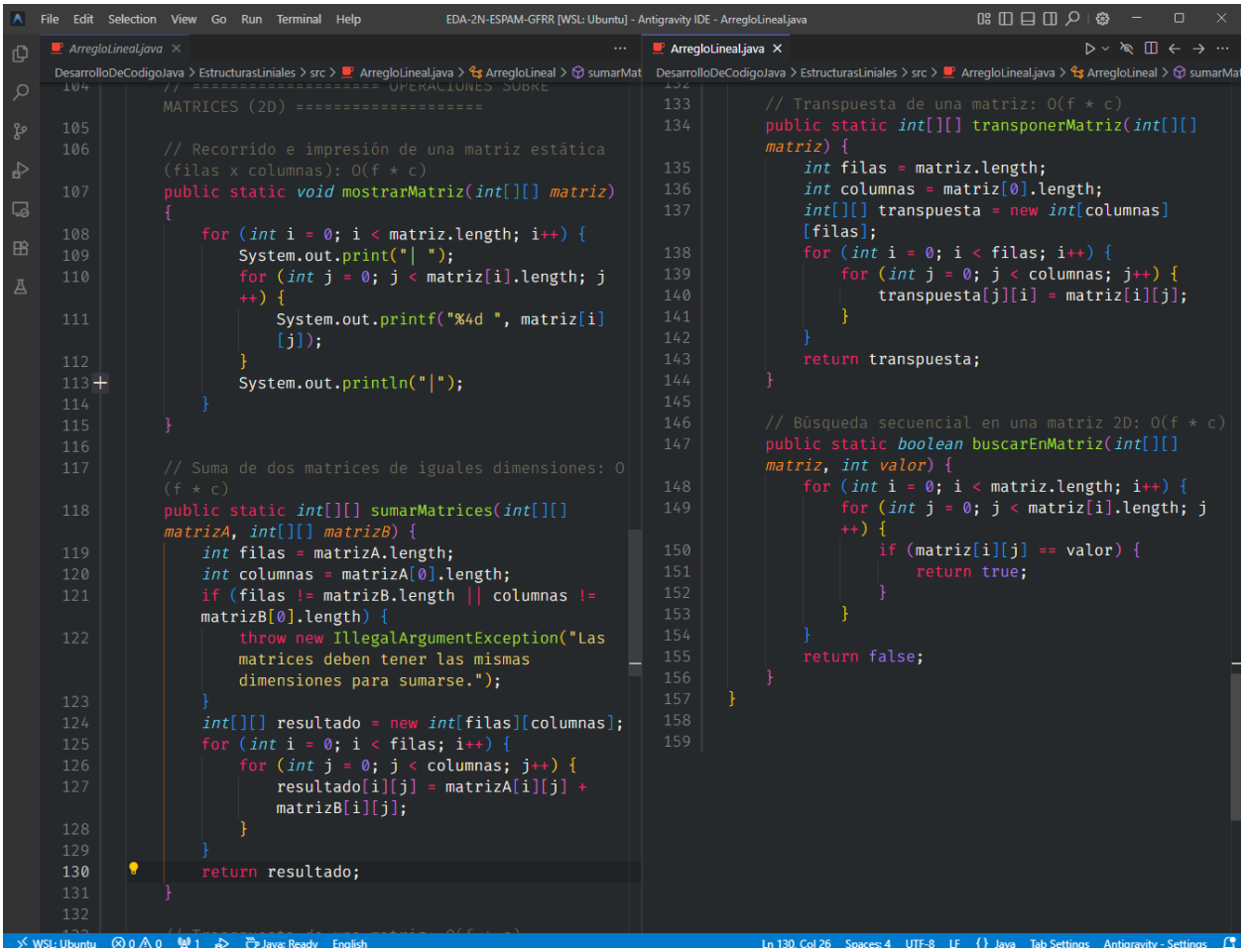
    // Avanza a la siguiente canción en O(1) (bucle
    continuo automático)
    public String siguiente() {
        if (actual == null) {
            throw new IllegalStateException("La
            playlist está vacía.");
        }
        actual = actual.siguiente; // O(1)
        return actual.cancion;
    }

    // Retrocede a la canción anterior en O(1)
    (bucle continuo inverso automático)
    public String anterior() {
        if (actual == null) {
            throw new IllegalStateException("La
            playlist está vacía.");
        }
        actual = actual.anterior; // O(1)
        return actual.cancion;
    }

    // Retorna la canción que se está reproduciendo
    actualmente: O(1)
    public String reproduciendoActual() {
        if (actual == null) {
            return "Ninguna (Playlist vacía)";
        }
        return actual.cancion;
    }
```

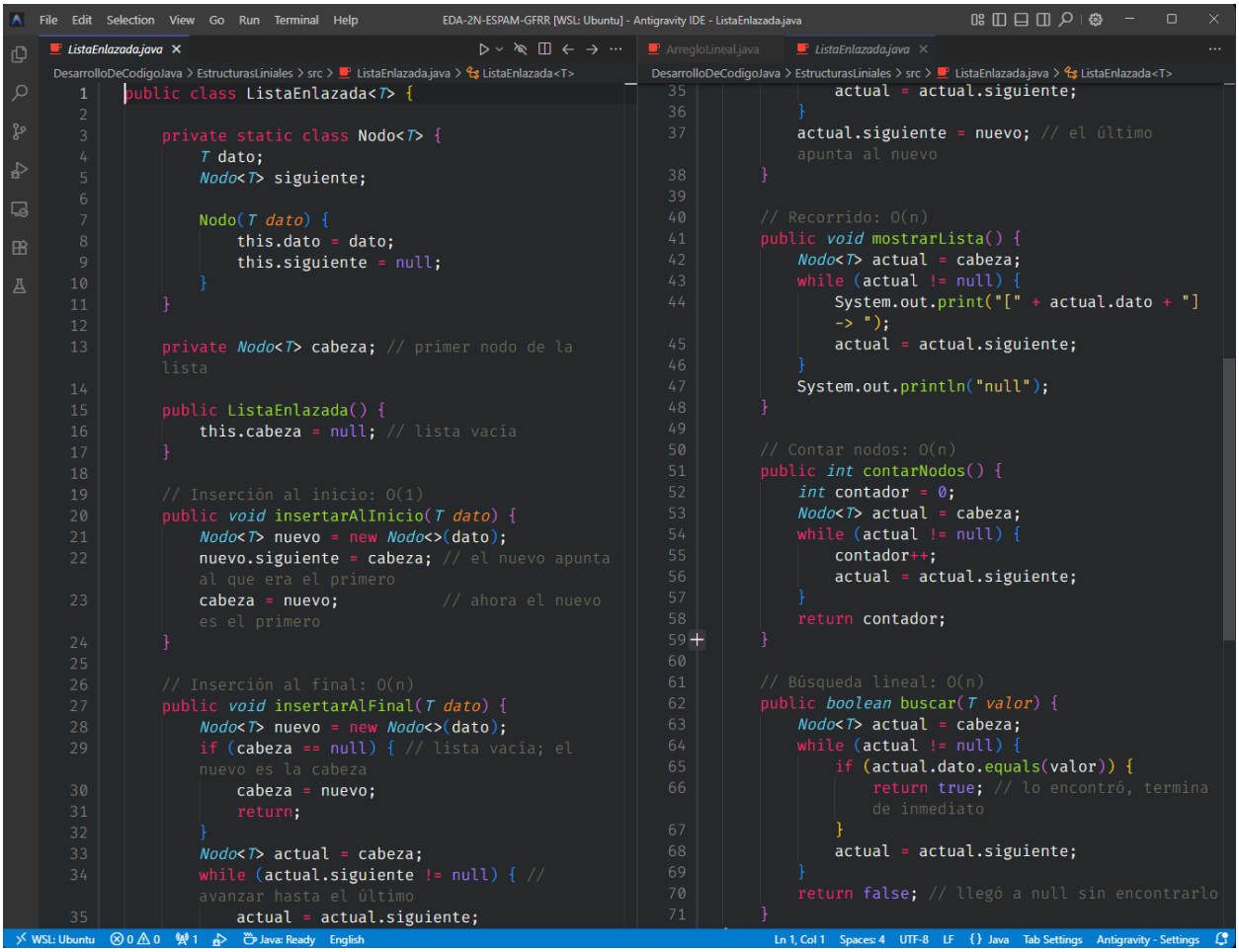
Comentario técnico de CasoForo.java: En CasoForo.java se define la clase estática interna Nodo (líneas 14–24) con la cadena cancion y las referencias siguiente y anterior. El método agregar(String cancion) (líneas 37–52) inserta nuevas canciones al final en tiempo constante O(1) sin recorrer la lista: obtiene el último nodo directamente mediante Nodo cola = cabeza.anterior y actualiza cuatro enlaces para cerrar el anillo (cola.siguiente = n, n.anterior = cola, n.siguiente = cabeza y cabeza.anterior = n). Los métodos siguiente() (líneas 55–61) y anterior() (líneas 64–70) desplazan el puntero actual en O(1), garantizando el bucle continuo en ambos sentidos sin encontrar jamás una referencia null.

B) Módulo de Arreglos Estáticos y Matrices (ArregloLineal.java): Implementa las operaciones fundamentales sobre arreglos estáticos unidimensionales (1D) y matrices bidimensionales (2D) en bloques contiguos de memoria.



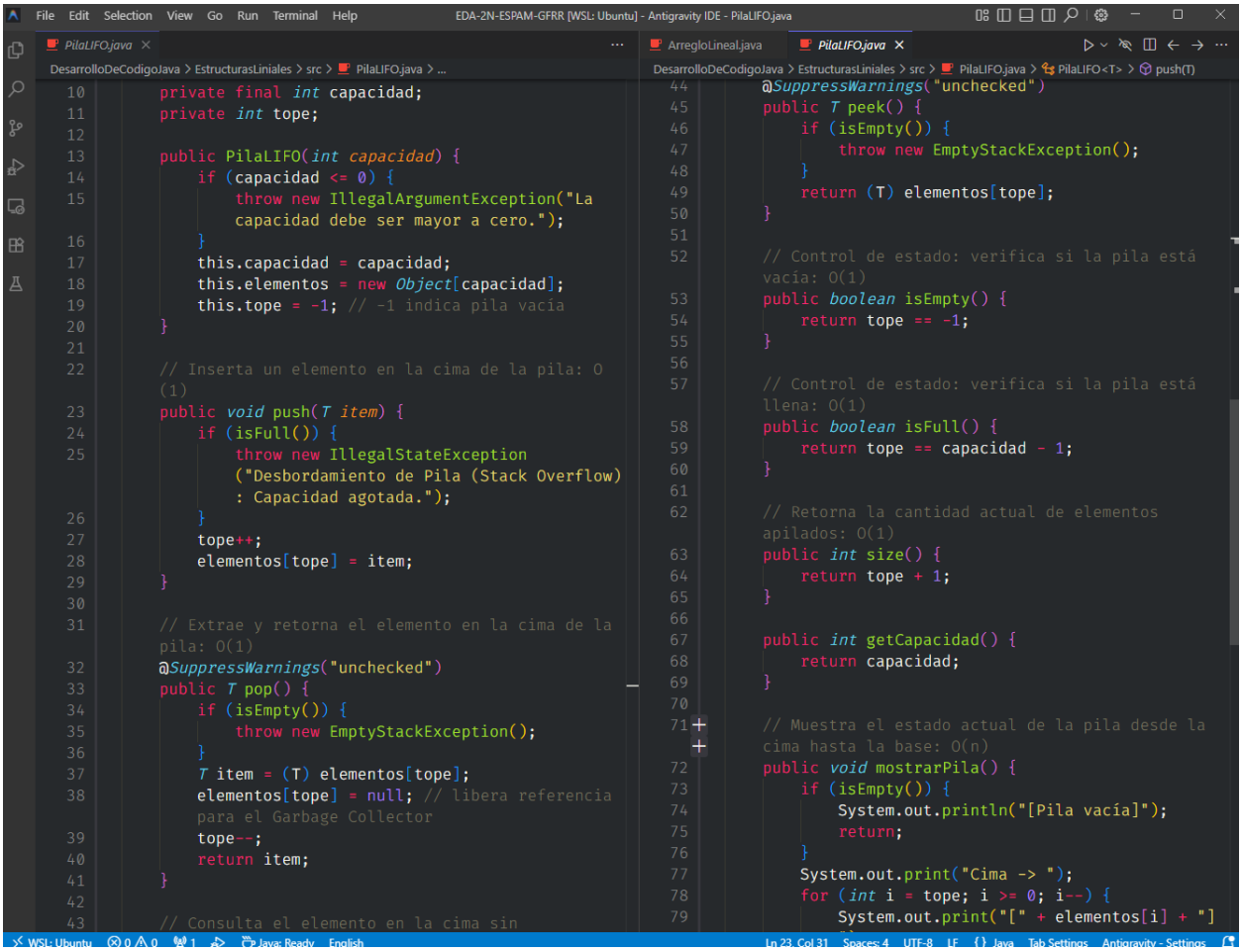
Comentario técnico de ArregloLineal.java: La clase ArregloLineal demuestra la principal ventaja de la memoria contigua: el método obtener(int indice) (líneas 45–50) y actualizar(int indice, int nuevoValor) (líneas 53–58) acceden de forma instantánea a cualquier celda en tiempo constante O(1) mediante cálculo directo de dirección base más desplazamiento. En contraste, insertarEnIndice() (líneas 30–42) y eliminarEnIndice() (líneas 61–72) evidencian el costo lineal O(n) derivado de desplazar físicamente los elementos hacia la derecha o izquierda. Adicionalmente, la sección de matrices bidimensionales (líneas 106–156) implementa el recorrido por filas y columnas, la suma matricial y la transposición en complejidad O(f * c).

C) Módulo de Lista Simplemente Enlazada (ListaEnlazada.java): Implementa una Lista Simplemente Enlazada genérica mediante nodos creados dinámicamente en el Heap.



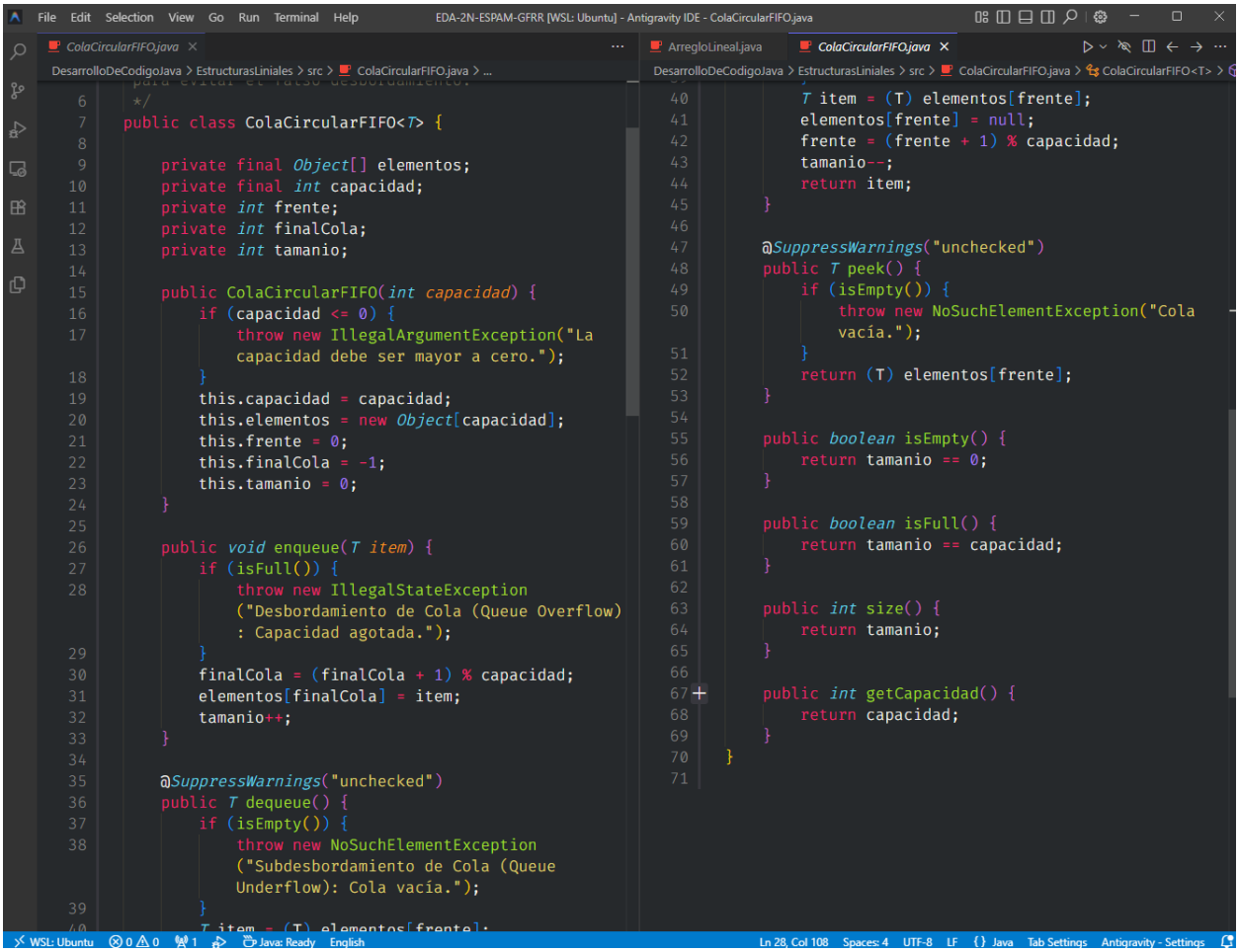
Comentario técnico de ListaEnlazada.java: En ListaEnlazada<T> cada elemento se aloja en el Heap a través de la clase interna Nodo<T> (líneas 3–11). El método insertarAlInicio(T dato) (líneas 20–24) demuestra la inserción eficiente en la cabeza en tiempo constante O(1) al hacer que nuevo.siguiente apunte a la antigua cabeza y actualizar cabeza = nuevo. Por su parte, insertarAlFinal(T dato) (líneas 27–38), mostrarLista() (líneas 41–48), contarNodos() (líneas 51–59) y buscar(T valor) (líneas 62–71) recorren secuencialmente las referencias actual = actual.siguiente hasta alcanzar null, operando en tiempo lineal O(n).

D) Módulo de Pila LIFO (PilaLIFO.java): Implementa el Tipo de Dato Abstracto Pila (Stack) bajo el principio LIFO (Last-In, First-Out) con control estricto de estados.



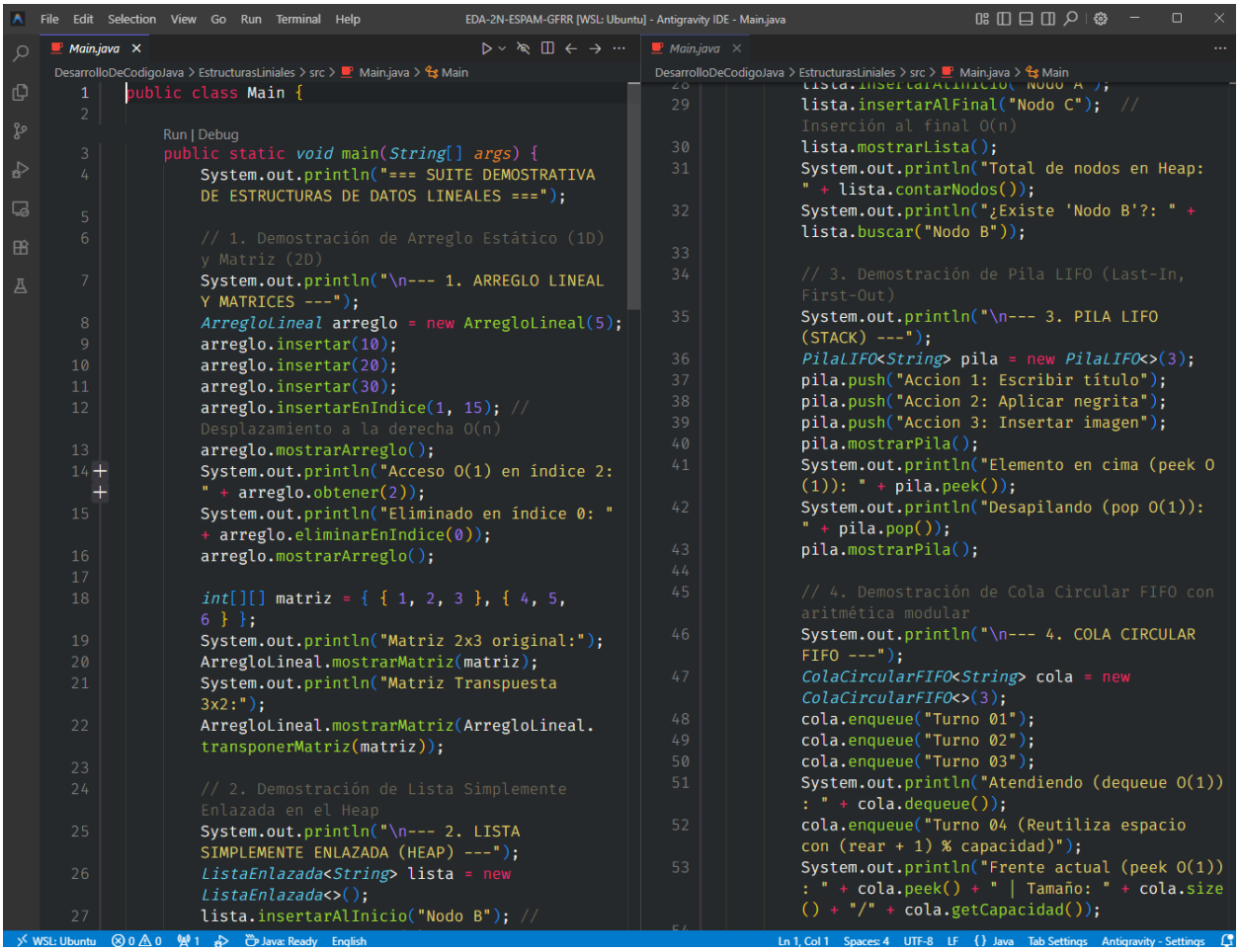
Comentario técnico de PilaLIFO.java: La clase PilaLIFO<T> administra las operaciones exclusivamente sobre el extremo superior identificado por el índice tope (inicializado en -1 cuando la pila está vacía). El método push(T item) (líneas 23–29) verifica primero la condición de desbordamiento mediante isFull() (Stack Overflow) antes de incrementar tope e insertar en O(1). El método pop() (líneas 33–41) valida el subdesbordamiento mediante isEmpty() (Stack Underflow / EmptyStackException), extrae el elemento superior en O(1), limpia la referencia en el arreglo (elementos[tope] = null) para evitar fugas de memoria en el Garbage Collector y decrementa tope. El método peek() (líneas 45–50) inspecciona la cima en O(1) sin modificar la estructura.

E) Módulo de Cola Circular FIFO (ColaCircularFIFO.java): Implementa una Cola Circular bajo política FIFO (First-In, First-Out) utilizando aritmética modular para reutilizar espacios desencolados.



Comentario técnico de ColaCircularFIFO.java: En una cola lineal simple basada en arreglo, las desencolaciones sucesivas desplazan el frente hacia la derecha dejando celdas vacías inutilizables al inicio (falso desbordamiento). ColaCircularFIFO<T> soluciona este problema aplicando aritmética modular en tiempo constante O(1): en el método enqueue(T item) (líneas 26–33), el puntero trasero avanza con la fórmula finalCola = (finalCola + 1) % capacidad (línea 30), dando la vuelta automáticamente al índice 0 cuando alcanza el final físico del arreglo si existen espacios libres. De forma simétrica, dequeue() (líneas 36–45) extrae el elemento del frente, libera la referencia y avanza con frente = (frente + 1) % capacidad (línea 42) en O(1).

F) Módulo Principal Demostrativo (Main.java) y Ejecución en Consola: Integra y ejecuta de manera secuencial todas las estructuras de datos desarrolladas junto con la simulación interactiva del caso del foro.



Análisis de los resultados en consola: La ejecución de Main.java comprueba empíricamente el funcionamiento integrado de toda la suite: (1) en ArregloLineal se verifica el acceso O(1) por índice, el desplazamiento en inserciones/eliminaciones y la transposición matricial; (2) en ListaEnlazada se comprueba la creación de nodos en el Heap y el enlace secuencial hasta null; (3) en PilaLIFO se valida la inversión cronológica LIFO mediante push, peek y pop; (4) en ColaCircularFIFO se demuestra cómo, tras desencolar el 'Turno 01', la inserción del 'Turno 04' reutiliza exitosamente el espacio libre mediante (rear + 1) % capacidad; y (5) en CasoForo se verifica que al avanzar después de la tercera pista ('3. Imagine - John Lennon') el reproductor retorna automáticamente a la primera pista ('1. Bohemian Rhapsody - Queen') en O(1), y al retroceder desde la primera vuelve instantáneamente a la última pista en O(1).

3. CONCLUSIÓN TÉCNICA

El desarrollo integral de este portafolio permite establecer conclusiones técnicas fundamentales sobre el diseño y selección de estructuras de datos lineales en Ingeniería de Software. En primer lugar, la comparativa entre estructuras de memoria contigua (ArregloLineal) y estructuras dinámicas enlazadas en el Heap (ListaEnlazada y CasoForo) demuestra que ninguna estructura es universalmente superior, sino que su idoneidad depende del patrón de acceso del dominio: cuando se requiere acceso aleatorio inmediato por posición [i][j] y el tamaño es conocido, los arreglos estáticos ofrecen complejidad O(1) sin sobrecosto de punteros; sin embargo, cuando el volumen de datos es elástico y predominan las inserciones, eliminaciones o navegaciones secuenciales/bidireccionales, las estructuras enlazadas en el Heap evitan la rigidez y el costo O(n) de desplazamiento físico de memoria.

Estructura Lineal	Disposición en Memoria	Acceso / Inspección	Inserción Principal	Eliminación / Avance
ArregloLineal (1D/2D)	Contigua en Heap	O(1) por índice [i]	O(1) final / O(n) medio	O(n) por desplazamiento
ListaEnlazada<T>	Nodos dispersos en Heap	O(n) secuencial	O(1) en cabeza / O(n) final	O(1) cabeza / O(n) medio
CasoForo (Doble Circular)	Nodos en anillo en Heap	O(1) pista actual	O(1) al final (cabeza.ant)	O(1) siguiente() / anterior()
PilaLIFO<T>	Contigua acotada (tope)	O(1) en cima (peek)	O(1) apilar (push)	O(1) desapilar (pop)
ColaCircularFIFO<T>	Anillo modular contiguo	O(1) en frente (peek)	O(1) modular (enqueue)	O(1) modular (dequeue)

Finalmente, la resolución del caso del Grupo 3 (Playlist Multimedia con Bucle Continuo) y el debate técnico en el foro evidenciaron cómo el costo de agregar un segundo puntero (anterior) por nodo y cerrar las referencias en anillo transforma operaciones que serían O(n) en una lista simple (como retroceder de pista o insertar en la cola) en operaciones estrictamente constantes O(1), logrando un diseño robusto, escalable y libre de excepciones por punteros nulos.